LoggerAppsMsg()

Synopsis

```
int   LoggerAppsMsg(devProd, message_group, severity, message_string, … )
int   devProd;
int   message_group;
int   severity;
char  *message_string;
```

Description

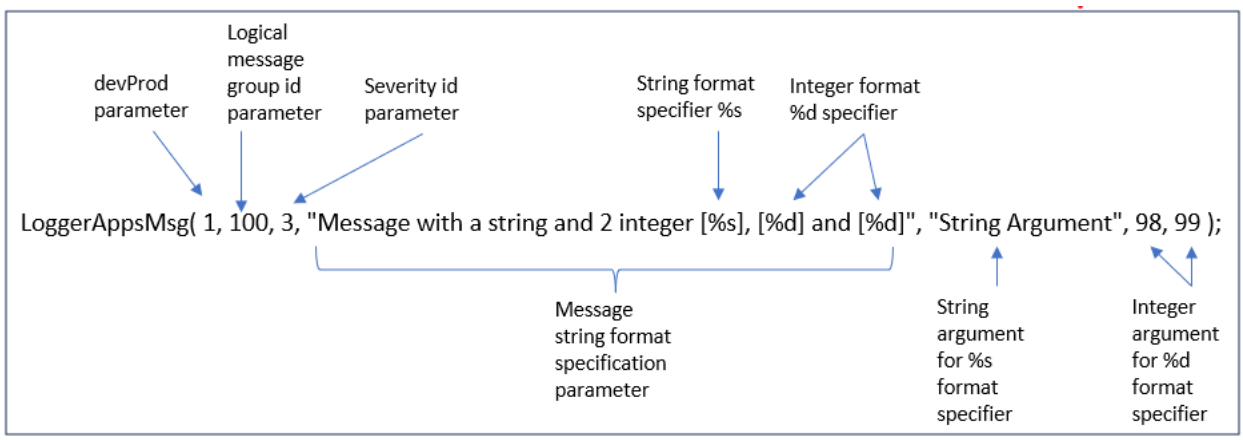
`LoggerAppsMsg()` decomposes argument parameters into id=value pairs, validates for errors, encode to reduce storage capacity, and securely protects to ensure confidentiality, integrity, and authenticity.[1]

Example Usage

```
LoggerAppsMsg(1,100,3,"Message that generates 1 string and 2 integers [%s],
[%d] and [%d]","String Argument", 98,99);
```

Generates formatted output log message:

```
02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFORM] Message that
generates 1 string and 2 integers [String Argument], [98] and [99]
```



The `devProd` integer parameter must be 1 or 2, where 1 means generate unprotected log messages.[2]

The logical `message_group` integer parameter is the id part of the id=value pairs defined in file LoggerMessageDefines.txt. The id part of 100 is translated and mapped to the corresponding value part of DATABASE in the output message. Refer to the LoggerMessageDefines.txt document for further details.

The `severity` integer parameter is the id part of the id=value pairs defined in file LoggerMessageDefines.txt. The id part of 3 is translated and mapped to the corresponding value part of INFORM in the output message. Refer to the LoggerMessageDefines.txt document for further details.

The `message_string` format string specification parameter is a C-Programming Language printf() style format string specification that comprises of format string text and format specifiers.

---

[1] The Free Version supports decompose and validate format string specification and string argument parameters.
[2] The Free Version supports devProd option 1 only.

The reminder of the `LoggerAppsMsg()` parameters (as indicated by "…") are the corresponding format string specifiers argument values for `message_string`.

NOTE: Refer to the Wandi-SASL Capabilities section Adjustable Parameters for validation guidelines used by `LoggerAppsMsg()` to check and validate parameters.

`LoggerAppsMsg()` returns 0 on success and -1 on error.

This API function is failed safe which means when an error occurs the application can continue processing.

Error Codes

Error codes are written to the following files, where the <cpid> is the application process id number:

- /tmp/<cpid>-LoggerAppsMessage.txt
- $LOGGER_HOME_PATH/LoggerOutput/<cpid>-LoggerAppsMessage.txt

Additional error codes detailed explanations are listed here.

1012	Invalid log env parameter.
1013	Invalid logical message grouping and/or severity id parameter.
1014	Invalid formatted message text.
1015	Too many format string specifiers.
1039	String argument pointer parameter is NULL.
1040	String argument pointer is negative.
1041	String argument not properly NULL terminator.
1016	String argument exceeded allowed length.
1017	Invalid string argument.
1018	Invalid format string specifier type.
1412, 1413, 1414	Invalid message text.
1400, 1401, 1403	Invalid character in message test or string argument.
1402	Invalid message string argument length of zero.